TRANSFORMER CHEAT SHEET



Follow for more

Naresh Edagotti

The complete visual guide to the architecture that changed AI forever — from Attention to modern variants.

ARCHITECTURE
From Attention to modern stack

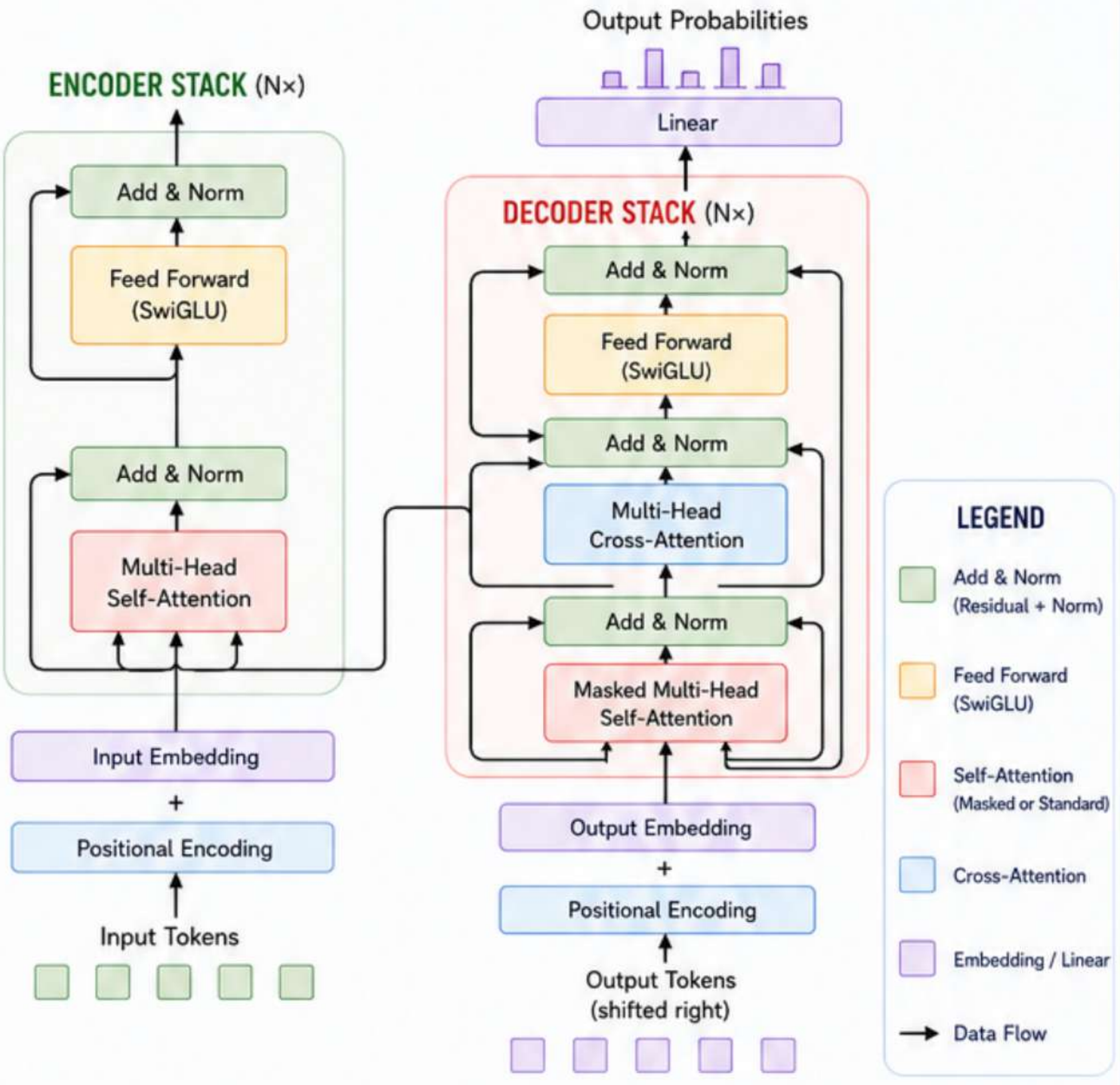
VARIANTS
Encoder, Decoder, MoE & beyond

VISION & MULTIMODAL
ViT, VLMs and multimodal models

CODE & MATH
Better for coding, reasoning & more

SCALING & TRAINING
Scaling laws, optimizers, precision & more

THE TRANSFORMER ARCHITECTURE (AT A GLANCE)



WHAT YOU'LL FIND INSIDE

Core Concepts
Clear, visual & simple

Architectures
All major variants

Training & Scaling
Optimizer, LR, scaling laws

Modern Advances
RoPE, GQA, MLA, Flash Attention & more

Vision & Multimodal
ViT, VLMs & beyond

Code & Math
Better reasoning, coding & math

Research Roadmap
Key papers to read

Attention • Encoder • Decoder • Positional Encoding • Multi-Head
Variants • Vision • Code • Math

THE ORIGIN STORY & BIG PICTURE

1 WHY TRANSFORMERS EXIST

Before Transformers, sequence modeling was dominated by **RNNs** and **LSTMs**. They had two fatal flaws:

1. **Sequential processing** — Each token had to wait for the previous one. Cannot parallelize. Slow training.

2. **Vanishing gradients** — Long-range dependencies degraded badly over distance. The model "forgot" what happened 50 tokens ago.

Transformers solved both problems in one shot with a single idea: replace recurrence with attention entirely.

> "Attention Is All You Need" — Vaswani et al., Google Brain, 2017
>
> This 8-page paper is the most cited paper in AI history.

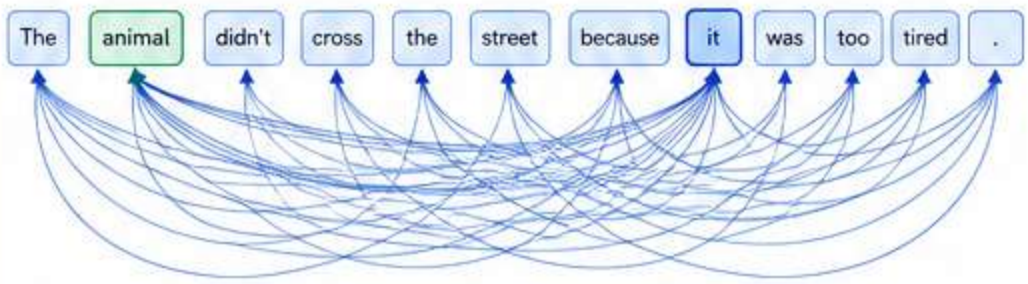
2 THE CORE INTUITION

Imagine reading a sentence:

> "The animal didn't cross the street because it was too tired."

What does "it" refer to? You don't read left to right like an RNN. You scan the whole sentence and attend to **"animal"** — because it makes semantic sense.

That's exactly what the Transformer does. **Every token looks at every other token simultaneously** and decides how much to "attend" to each one.



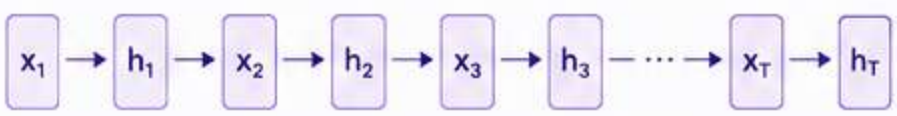
3 WHAT TRANSFORMERS REPLACED

Old Architecture	Problem	Transformer Solution
RNN	Sequential, slow, can't parallelize	Fully parallel attention ✓
LSTM	Better memory but still sequential	Replaced entirely ✓
CNN for sequences	Fixed receptive field	Global attention from step 1 ✓
Attention + RNN (hybrid)	RNN still bottleneck	Pure attention, no RNN ✓

4 WHERE TRANSFORMERS ARE USED TODAY

NLP	→	GPT, BERT, T5, LLaMA, Claude, Gemini
Vision	→	ViT, DINO, Stable Diffusion (U-Net with attention)
Audio	→	Whisper, AudioLM, MusicGen
Code	→	Codex, StarCoder, DeepSeek Coder
Multimodal	→	GPT-4V, Flamingo, LLaVA, Gemini
Protein	→	AlphaFold 2 (Evoformer)
Reinforcement Learning	→	Decision Transformer, Gato
Time Series	→	Temporal Fusion Transformer, PatchTST

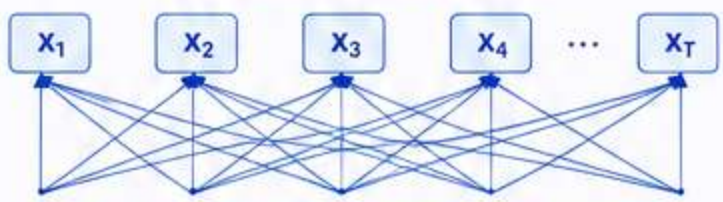
RNN: One step at a time



Slow, sequential, hard to parallelize

VS

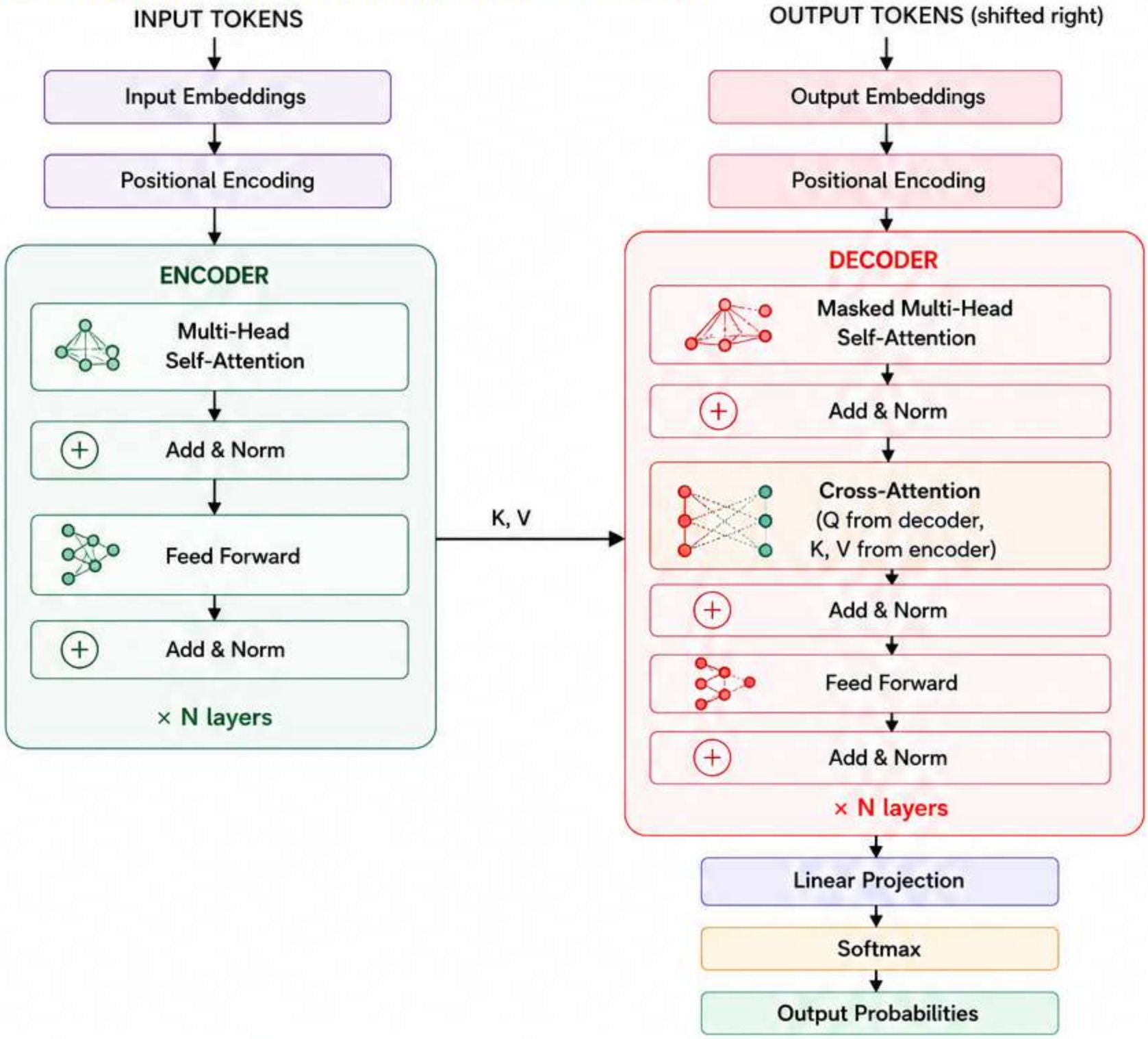
Transformer: All at once



Fully parallel, captures long-range dependencies

FULL ARCHITECTURE OVERVIEW

1 The Original Transformer (Encoder-Decoder)



2 Three Transformer Flavors

Type	Architecture	Examples	Best For
Encoder-Only		BERT, RoBERTa, ALBERT, DeBERTa	Classification, embeddings, NER, understanding
Decoder-Only		GPT-4, LLaMA, Mistral, Claude	Text generation, chat, completion
Encoder-Decoder		T5, BART, mT5, NLLB	Translation, summarization, QA

3 Model Dimensions — Key Hyperparameters

Hyperparameter	Symbol	Typical Values	What it Controls
Model dimension	d_{model}	512, 768, 1024, 4096	Width of every embedding vector
Number of layers	N	6, 12, 24, 32, 80	Depth of the model
Attention heads	h	8, 12, 16, 32, 64	Parallel attention subspaces
Head dimension	$d_k = d_{model} / h$	64, 128	Dimension per attention head
FFN dimension	d_{ff}	$4 \times d_{model}$	Size of the FFN hidden layer
Vocab size	V	32K, 50K, 100K, 128K	Number of unique tokens
Max sequence length	L	512 → 1M+	Context window

ATTENTION MECHANISM — DEEP DIVE

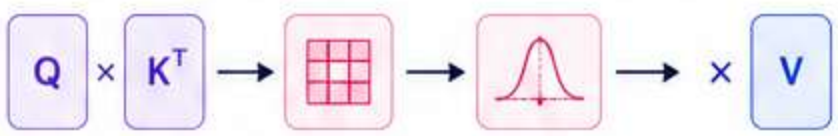
1 THE ATTENTION FORMULA

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \cdot V$$

★ This is the most important equation in modern AI. Break it down:

Symbol	Shape	Meaning
Q (Query)	$[\text{seq_len} \times d_k]$	What this token is looking for
K (Key)	$[\text{seq_len} \times d_k]$	What each token announces it has
V (Value)	$[\text{seq_len} \times d_v]$	The actual content each token carries
QK^T	$[\text{seq_len} \times \text{seq_len}]$	Attention score matrix — every token vs every token
$/\sqrt{d_k}$	scalar	Scaling to prevent extremely peaked softmax
softmax(·)	$[\text{seq_len} \times \text{seq_len}]$	Convert scores to probability distribution
$\cdot\ V$	$[\text{seq_len} \times d_v]$	Weighted sum of values = attended output

What happens step-by-step?



1. **Dot Product (QK^T)** — Compare every query with every key
2. **Scale** — Divide by $\sqrt{d_k}$ to keep values in a stable range
3. **Softmax** — Convert scores to probabilities (rows sum to 1)
4. **Weighted Sum** — Multiply by V to get the final attended representation

Shapes at a glance (single head)

$Q : [\text{seq_len}, d_k]$ $K : [\text{seq_len}, d_k]$ $V : [\text{seq_len}, d_v]$

$QK^T : [\text{seq_len}, \text{seq_len}]$ $\text{softmax}(\ldots) : [\text{seq_len}, \text{seq_len}]$

Attention Output : $[\text{seq_len}, d_v]$

2 WHY SCALE BY $\sqrt{d_k}$?

Without scaling, dot products grow large as d_k increases. Large values push softmax into saturation regions where gradients vanish.

Example:

$d_k = 64 \rightarrow \sqrt{d_k} = 8$	logits stay in a reasonable range ✓
$d_k = 512 \rightarrow \sqrt{d_k} \approx 22.6$	prevents logits from exploding to 100+ ✓

Scaling keeps the softmax in the sensitive region, leading to better gradients and faster learning.

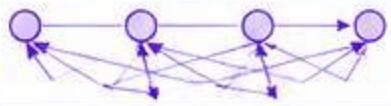
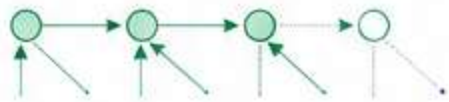
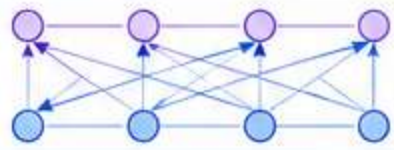
3 ATTENTION SCORE MATRIX — WHAT IT LOOKS LIKE

	Token 1	Token 2	Token 3	Token 4
Token 1	0.70	0.15	0.10	0.05
Token 2	0.05	0.80	0.10	0.05
Token 3	0.20	0.20	0.50	0.10
Token 4	0.05	0.10	0.15	0.70

High Attention (1.0) … Low Attention (0.0)

- Each row sums to 1.0 (after softmax)
- Each row = "how much does this token attend to each other token"

4 THREE TYPES OF ATTENTION

Type	Used In	Q comes from	K, V come from	Mask?	Illustration
Self-Attention	Encoder	Same sequence	Same sequence	None (full)	
Masked Self-Attention	Decoder	Same sequence	Same sequence	Causal mask	
Cross-Attention	Decoder	Decoder sequence	Encoder output	None	

5 CAUSAL MASKING (DECODER SELF-ATTENTION)

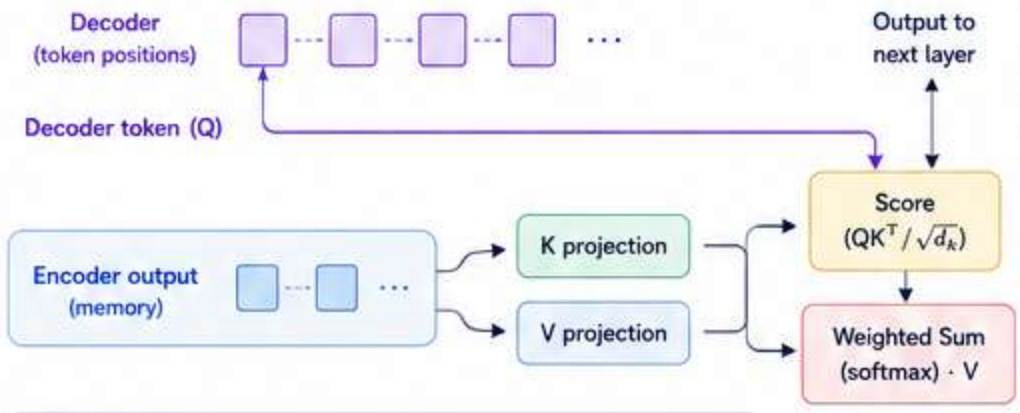
	t1	t2	t3	t4
t1	1.0	−∞	−∞	−∞
t2	0.3	0.7	−∞	−∞
t3	0.2	0.3	0.5	−∞
t4	0.1	0.2	0.3	0.4

−∞ becomes 0 after softmax

Token at position t can only attend to positions $\leq t$

This ensures autoregressive generation — no future peeking

6 CROSS-ATTENTION — HOW DECODER READS ENCODER



The decoder asks: "Given what I've generated so far, which parts of the input sequence are most relevant?"

MULTI-HEAD ATTENTION

1 WHY MULTIPLE HEADS?

A single attention head captures one type of relationship at a time. Multiple heads let the model simultaneously attend to:

- **Syntactic dependencies** (subject-verb agreement)
- **Coreference** (pronoun → noun)
- **Semantic similarity** (synonyms)
- **Positional proximity** (local context)

★ Each head learns different, complementary relationships in parallel.

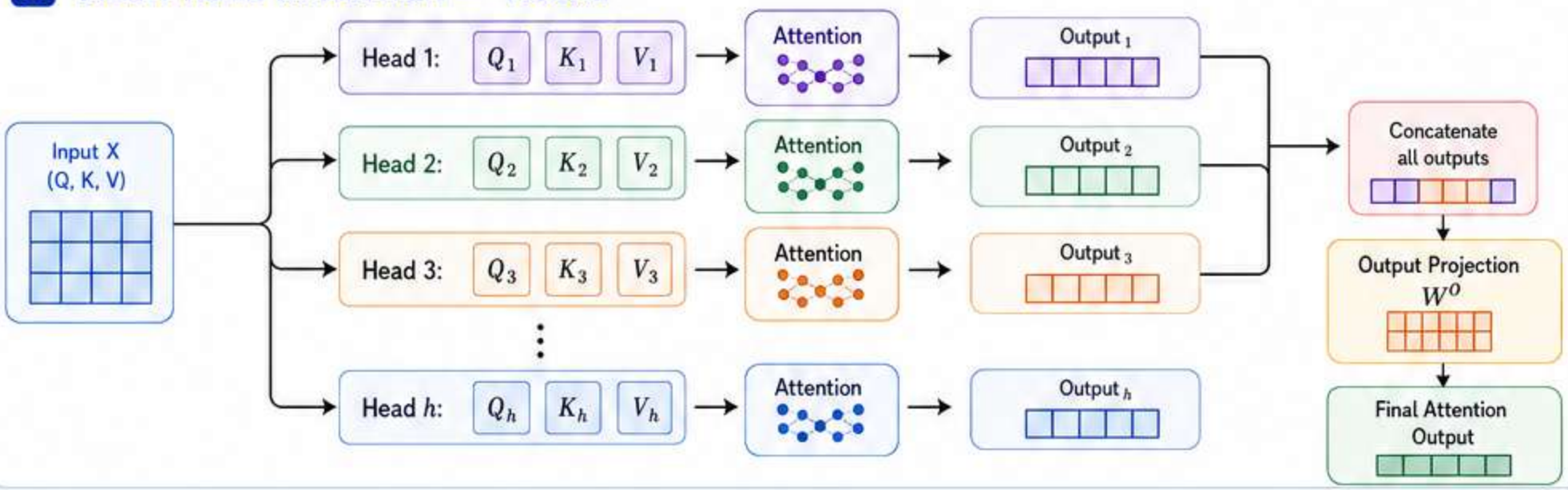
2 MULTI-HEAD ATTENTION FORMULA

$$\text{head}_i = \text{Attention}(QW_i^Q,\ KW_i^K,\ VW_i^V)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(head_1,\ head_2,\ \ldots,\ head_h) \cdot W^O$$

Variable	Description
W_i^Q, W_i^K, W_i^V	Learned projection matrices per head
head_i	Output of the i-th attention head
Concat	All heads concatenated along feature dimension
W^O	Final output projection matrix

3 MULTI-HEAD ATTENTION — VISUAL



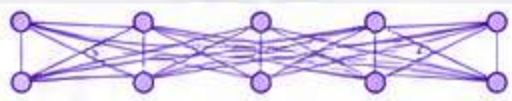
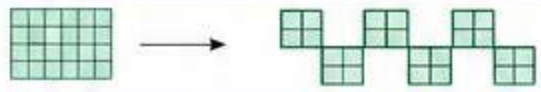
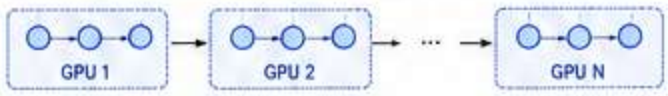
4 GROUPED QUERY ATTENTION (GQA) & MULTI-QUERY ATTENTION (MQA)

Modern LLMs use these variants to reduce KV cache memory:

Variant	Heads for Q	Heads for K, V	Memory	Quality
MHA (standard)	H	H	Largest	Best
MQA	H	1 (shared)	Smallest	Slight drop
GQA	H	G $(1 < G < H)$	Middle	Near-MHA

★ LLaMA 2 70B, Mistral, Gemma → all use GQA
GQA is now the industry standard for production LLMs

5 EFFICIENT ATTENTION VARIANTS

Variant	Idea	Complexity	Visual Intuition
Vanilla Attention	All-to-all attention	$O(n^2)$	
Flash Attention	IO-aware tiling on GPU, no approximation	$O(n^2)$ but fast in practice	
Flash Attention 2 / 3	Better parallelism, 2–3× faster than FA1	$O(n^2)$	
Sparse Attention	Attend to fixed patterns only	$O(n\sqrt{n})$	
Sliding Window Attention	Attend to local window only	$O(n \cdot w)$	
Longformer	Local + global token attention	$O(n)$	
Linear Attention	Kernel trick to avoid explicit QK^T	$O(n)$	
Ring Attention	Distribute sequence across devices	$O(n\ /\ devices)$	

POSITIONAL ENCODING

1 WHY POSITION MATTERS

Attention is permutation invariant — shuffle the tokens and attention scores stay the same. The model has no built-in sense of order. Position encoding injects order information into the embeddings.

"Dog bites man" ≠ "Man bites dog"

⚠ Without positional encoding → Transformer sees these as identical

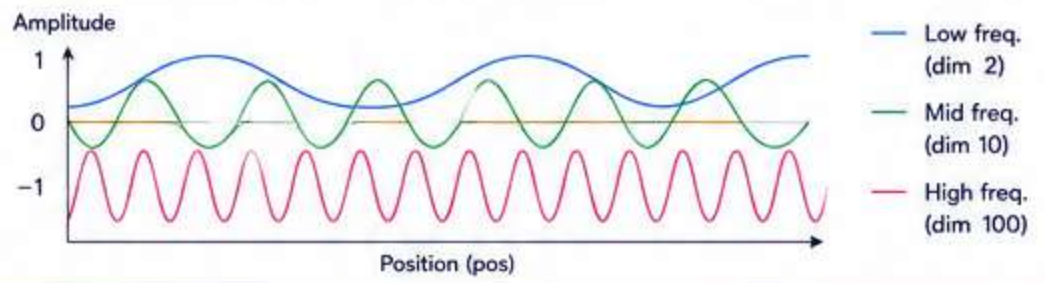
2 SINUSOIDAL POSITIONAL ENCODING (ORIGINAL)

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

- Fixed — not learned, computed from formula
- Different frequency for each dimension
- Allows the model to extrapolate to unseen lengths
- Still used in some architectures today

💡 Intuition: Think of it like a binary clock where each dransformrer oscillates at a different frequency.



3 POSITIONAL ENCODING TYPES — FULL COMPARISON

Type	Learned?	Relative?	Extrapolates?	Used In	Notes
Sinusoidal (Original)	No	No	Partially (long but not optimal)	Original Transformer (Vaswani et al., 2017)	Fixed formula
Learned Absolute	Yes	No	No	BERT, GPT-2	Learned position embeddings
Relative (Shaw et al.)	Yes	Yes	Yes	Transformer-XL	Relative distance bias
RoPE (Rotary)	No	Yes	Yes (with tweaks)	LLaMA, Mistral, GPT-NeoX	Rotation-based, distance aware
ALiBi (Linear Bias)	No	Yes	Yes	BLOOM, MPT	Linear bias on attention scores
NoPE (None)	None	None	N/A	Some modern variants	Rely on other inductive biases
xPos (Scaled RoPE)	No	Yes	Yes	LongRoPE	Better extrapolation with scaling

4 ROPE — ROTARY POSITION EMBEDDING (DEEP DIVE)

RoPE is the dominant positional encoding in modern LLMs. Instead of adding position info to embeddings, it rotates the query and key vectors by an angle proportional to their position.

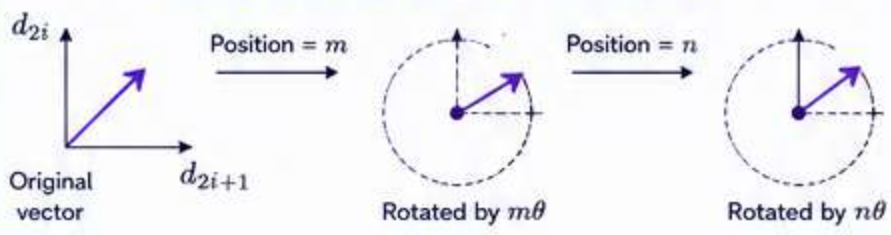
Key insight: The dot product between Q at position m and K at position n depends only on their relative distance $(m - n)$, not absolute positions.

Rotation matrix $R(\theta_m)$ applied to pairs of dimensions:

$$R(\theta_m) = \begin{bmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{bmatrix} \begin{bmatrix} q_1 \\ q_2 \end{bmatrix} \longrightarrow \begin{bmatrix} q_1' \\ q_2' \end{bmatrix}$$

where $\theta = 10000^{-2i/d_{model}}$ for dimension pair i

Rotate Q and K by position-dependent angle



Properties:

- ✓ Relative position awareness built into dot products
- ✓ Works perfectly with KV cache
- ✓ Generalizes to longer contexts with YaRN, LongRoPE, ABF scaling

5 ALIBI — ATTENTION WITH LINEAR BIASES

Instead of adding position to embeddings, ALiBi adds a linear bias directly to attention scores:

$$\text{Attention score} = \frac{QK^T}{\sqrt{d_k}} - m \cdot |i - j|$$

where m is a head-specific slope, and $|i - j|$ is token distance

Example: bias matrix added to attention logits

Key position j (columns)

Query position i (rows)	0	1	2	3	4	5	⋯
0	0	-m	-2m	-3m	-4m	-5m	⋯
1	-m	0	-m	-2m	-3m	-4m	⋯
2	-2m	-m	0	-m	-2m	-3m	⋯
3	-3m	-2m	-m	0	-m	-2m	⋯
4	-4m	-3m	-2m	-m	0	-m	⋯
5	-5m	-4m	-3m	-2m	-m	0	⋯
⋮	⋮	⋮	⋮	⋮	⋮	⋮	

Higher penalty ↓ Lower penalty

- ✓ No learned parameters
- ✓ Naturally penalizes attending to distant tokens
- ✓ Extrapolates well to longer sequences than trained on

FEED FORWARD NETWORK & LAYER NORM

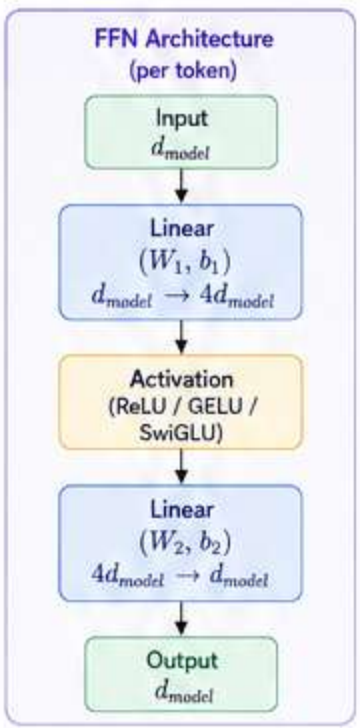
1 FEED FORWARD NETWORK (FFN)

After attention, every token passes independently through a Feed Forward Network:

$$FFN(x) = Activation(xW_1 + b_1) \cdot W_2 + b_2$$

Parameter	Typical Value
Input dim	d_{model} (e.g., 4096)
Hidden dim	$4 \times d_{model}$ (e.g., 16384)
Output dim	d_{model} (e.g., 4096)
Activation	ReLU (original) / GELU / SwiGLU

Key insight: While attention mixes information across tokens, FFN processes each token independently. Research shows FFN layers store factual knowledge.



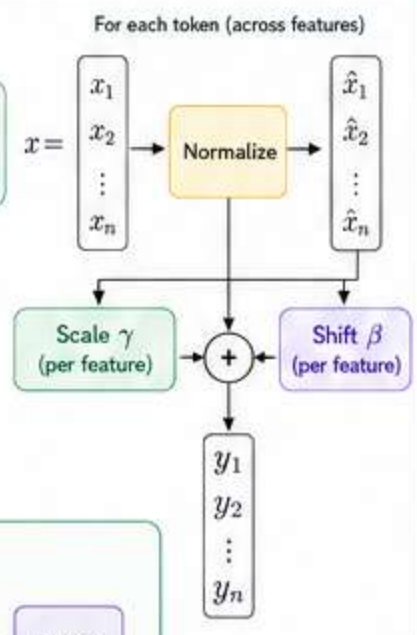
2 ACTIVATION FUNCTIONS

Activation	Formula	Used In	Notes
ReLU	$\max(0, x)$	Original Transformer	Simple, dead neurons problem
GELU	$x \cdot \Phi(x)$	BERT, GPT-2	Smooth, probabilistic
SiLU / Swish	$x \cdot \sigma(x)$	Many modern models	Better than ReLU
SwiGLU	$\text{Swish}(xW) \cdot (xV)$	LLaMA, PaLM, GPT-4	State-of-the-art FFN
GeGLU	$GELU(xW) \cdot (xV)$	T5 variants	Gated version of GELU

SwiGLU uses three weight matrices (vs two for standard FFN) and a gating mechanism. Consistently outperforms ReLU and GELU across scales.

3 LAYER NORMALIZATION

$$LayerNorm(x) = \gamma \cdot \frac{x - \mu}{\sigma + \epsilon} + \beta$$



μ = mean across features

σ = std deviation across features

γ, β = learned scale and shift parameters

ϵ = small constant for numerical stability (1e-5)

Pre-Norm vs Post-Norm

POST-NORM (original 2017):

x → Sublayer → Add (x, output) → Norm

PRE-NORM (modern standard):

x → Norm → Sublayer → Add (x, output)

- Pre-Norm is more stable for deep networks and allows higher LR
- Almost all modern LLMs (LLaMA, GPT-4, Mistral) use Pre-Norm

4 RMSNORM — SIMPLER AND FASTER

$$RMSNorm(x) = \gamma \cdot \frac{x}{RMS(x)} \qquad RMS(x) = \sqrt{\frac{1}{n}\sum_{i=1}^{n} x_i^2}$$

LayerNorm

1. Subtract mean: $x - \mu$
2. Divide by std: $\frac{x - \mu}{\sigma + \epsilon}$
3. Scale and shift: $\gamma \cdot (\cdot) + \beta$

VS

RMSNorm

1. No mean-centering: x
2. Divide by RMS: $\frac{x}{RMS(x)}$
3. Scale (no shift): $\gamma \cdot (\cdot)$

- Drops the mean-centering step (re-centering)
- 10–20% faster than LayerNorm
- Works just as well empirically
- Used in: LLaMA, Mistral, Gemma, Qwen, Falcon

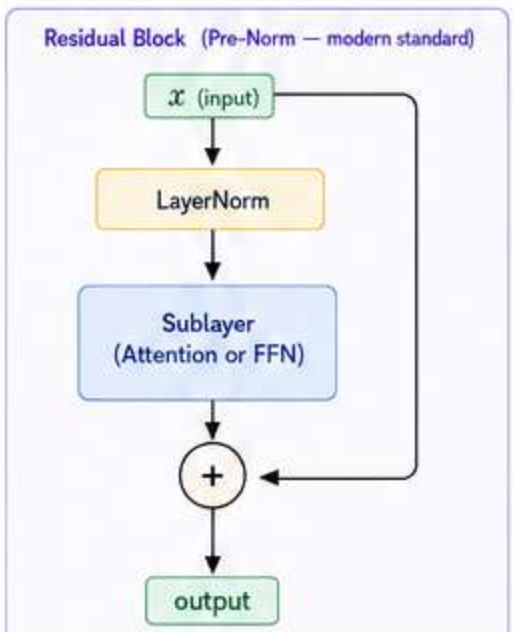
5 RESIDUAL CONNECTIONS

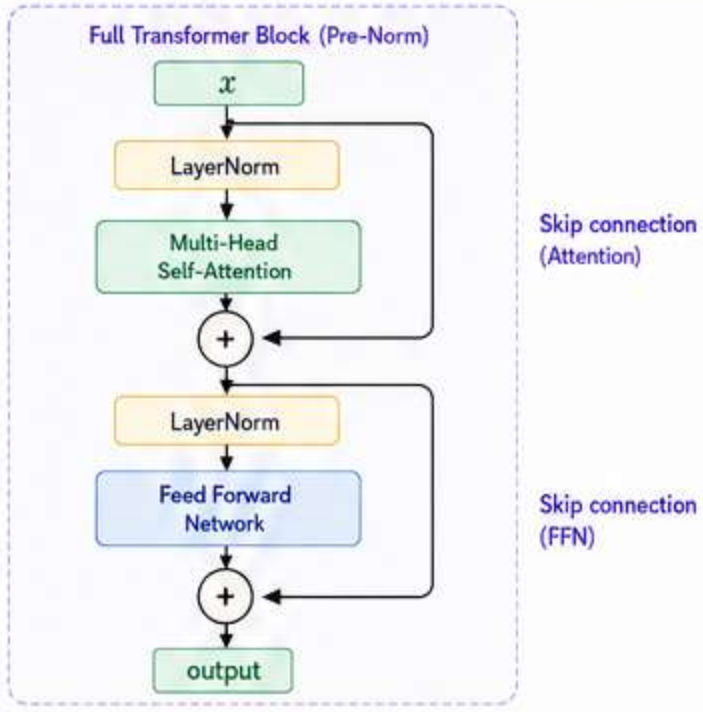
Every sub-layer (attention + FFN) has a residual (skip) connection:

$$output = LayerNorm(x + Sublayer(x))$$

Why it matters:

- Gradients flow directly through skip connections → solves vanishing gradients
- Allows training very deep networks (100+ layers)
- The network only needs to learn the residual (the delta), not the full transformation
- Identity mapping is the easy solution — the network learns deviations from it





TRAINING THE TRANSFORMER

1 TRAINING OBJECTIVE

For Decoder-Only (LLMs):
Next Token Prediction
(Causal Language Model)

$$\text{Loss} = -\frac{1}{T}\sum_{i=1}^{T}\log P(t_i \mid t_1, t_2, \ldots, t_{i-1})$$

Predict the next token given all previous tokens.

For Encoder-Only (BERT):
Masked Language Model (MLM)

- Randomly mask 15% of tokens → predict the masked tokens
- Loss = Cross-entropy on masked positions only

Input: The [MASK] cat sat on the [MASK] .
(Predict) (Predict)

For Encoder-Decoder (T5):
Span Corruption

- Mask contiguous spans of text → predict the masked spans

Input: This is a great [span to mask] example .
(Predict)

2 OPTIMIZER

AdamW is the universal standard for transformer training.

Adam:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \quad \text{(momentum)}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \quad \text{(adaptive LR)}$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$\theta = \theta - \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

AdamW adds decoupled weight decay:

$$\theta = \theta - \alpha \cdot \left[\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda\theta\right]$$

Common hyperparameters:

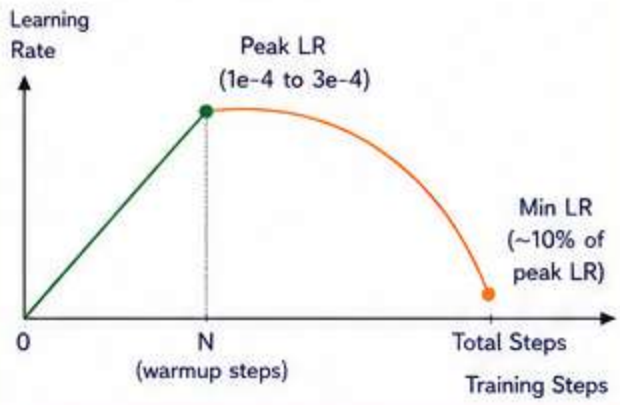
β_1	β_2	ϵ	λ
0.9	0.95 (not 0.999)	1e-8	0.1

3 LEARNING RATE SCHEDULE

Two-phase schedule is the standard.

Phase 1 — Warmup (linear)
0 → peak LR over first N steps

Phase 2 — Decay (cosine)
peak LR → min LR over training



Typical values:

- Peak LR: 1e-4 to 3e-4
- Min LR: ~10% of peak LR
- Warmup: 1000–4000 steps

4 REGULARIZATION TECHNIQUES

Technique	Where Applied	Typical Value
Dropout	Attention weights, FFN, embeddings	0.1 (smaller for larger models)
Weight Decay	All weight matrices (not biases/norms)	0.1
Gradient Clipping	Gradient norm	max_norm = 1.0
Label Smoothing	Output cross-entropy loss	0.1
Stochastic Depth	Randomly skip layers during training	Used in ViT variants

These techniques prevent overfitting and improve generalization, especially at large scale.

5 MIXED PRECISION TRAINING

Weights: stored in FP32 (master copy)

Forward: computed in FP16 or BF16 (fast, low memory)

Backward: gradients in FP16/BF16

Update: accumulated in FP32 master copy

BF16 vs FP16:

FP16
= 1 sign + 5 exponent + 10 mantissa
→ overflow risk

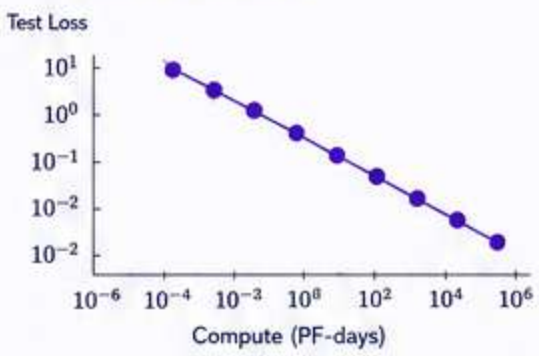
BF16
= 1 sign + 8 exponent + 7 mantissa
→ same range as FP32, more stable

BF16 is preferred for LLM training.

6 SCALING LAWS (CHINCHILLA)

Kaplan et al. (2020) — OpenAI

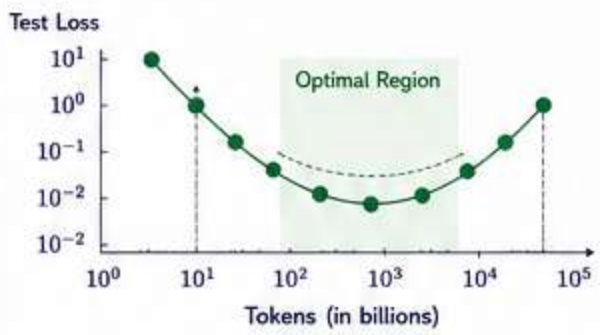
- Loss scales as power law with compute, data, and parameters
- Bigger models are always better (given enough compute)



More compute → lower loss (power law).

Chinchilla (Hoffmann et al., 2022) — DeepMind

- Optimal: train on 20 × tokens per parameter
- 70B model → needs ~1.4T tokens (not 300B like Gopher)
- Compute-optimal ≠ largest possible model



Too little data → underfit | Too much data → waste compute

Chinchilla rule of thumb:

$$N_{\text{optimal}} = C^{0.5}$$
(parameters scale with $\sqrt{\text{compute}}$)

$$D_{\text{optimal}} = C^{0.5}$$
(data scales with $\sqrt{\text{compute}}$)

$$\text{Tokens} \approx 20 \times \textit{Parameters}$$

C = total compute (FLOPs)

Key takeaway: More compute, more data, and more parameters reduce loss — but the optimal balance matters.
Train on ~20 tokens per parameter for compute-optimal models.

PAGE 9 – POSITIONAL ENCODING & CONTEXT LENGTH EXTENSION

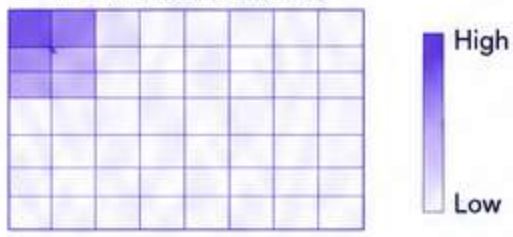
1 THE CONTEXT LENGTH PROBLEM

Standard transformers struggle beyond their training context length.

Attention complexity is $O(n^2)$ — doubling context **quadruples** compute and memory.

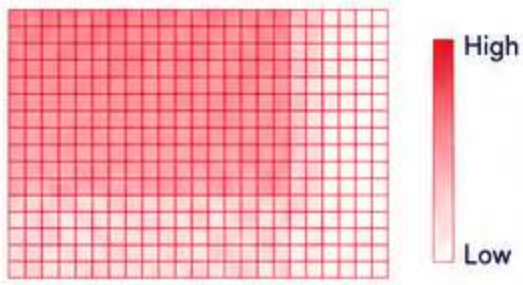
4K context

$4K^2 = 16M$
attention operations



128K context

$128K^2 = 16B$
attention operations
(1000× more)



Longer context requires algorithmic and engineering breakthroughs to be practical.

2 CONTEXT EXTENSION TECHNIQUES

Technique	How it Works	Used In
Position Interpolation	Rescale position IDs to fit new length	Extended LLaMA models
YaRN	Combines interpolation + NTK scaling	Mistral 32K, many fine-tunes
LongRoPE	Non-uniform position rescaling	Microsoft LongRoPE models
ABF (Adjusted Base Frequency)	Change RoPE base from 10K to 500K+	LLaMA 3, CodeLLaMA
Ring Attention	Split sequence across GPUs in a ring	1M+ context training
Sliding Window	Each token attends to local window only	Mistral 7B
Sparse Attention	Selective token attention patterns	Longformer, BigBird

3 FLASH ATTENTION — THE ENGINEERING BREAKTHROUGH

Standard attention stores the full $N \times N$ attention matrix in HBM (slow GPU memory):

STANDARD ATTENTION

Load Q, K, from HBM → Compute QK^T → Store to HBM → Load → Softmax → Store to HBM → Load V → Output

FLASH ATTENTION

Load Q, K, V in tiles → Compute attention block-by-block in SRAM (on-chip memory) → Output directly (no large matrix)

RESULT:

- No $N \times N$ matrix ever stored in HBM
- 2–4× faster than standard attention
- Exact (not approximate) — no quality loss
- Linear memory in sequence length

Flash Attention is exact, not approximate, and delivers massive speed and memory savings.

TRANSFORMER VARIANTS

1 ENCODER-ONLY MODELS

- Pre-trained with MLM + Next Sentence Prediction
- Sees the full sequence in both directions (no causal mask)
- Not for generation — excellent for understanding tasks

Model	Params	Context	Innovation
BERT-base	110M	512	Original MLM pretraining
RoBERTa	125M	512	Remove NSP, train longer, more data
ALBERT	12M	512	Parameter sharing across layers
DeBERTa	140M	512	Disentangled attention, relative positions
ModernBERT	149M	8192	2024 BERT with modern techniques

2 DECODER-ONLY MODELS

- Causal (autoregressive) — generates left to right
- Optimized for generation, reasoning, coding, chat
- Most LLMs used today are decoder-only

Model	Params	Context	Key Innovation
GPT-2	1.5B	1024	Showed generation at scale
GPT-3	175B	2048	Few-shot learning emergent
LLaMA 2	7B–70B	4096	Open weights, GQA in 70B
LLaMA 3	8B–405B	128K	Strong multilingual, long context
Mistral 7B	7B	32K	Sliding window + GQA, tiny but powerful
Mixtral 8×7B	47B (12.9B active)	32K	Mixture of Experts
Qwen 2.5	0.5B–72B	128K	Strong code + math
DeepSeek V3	671B (37B active)	128K	MLA + MoE, matches GPT-4

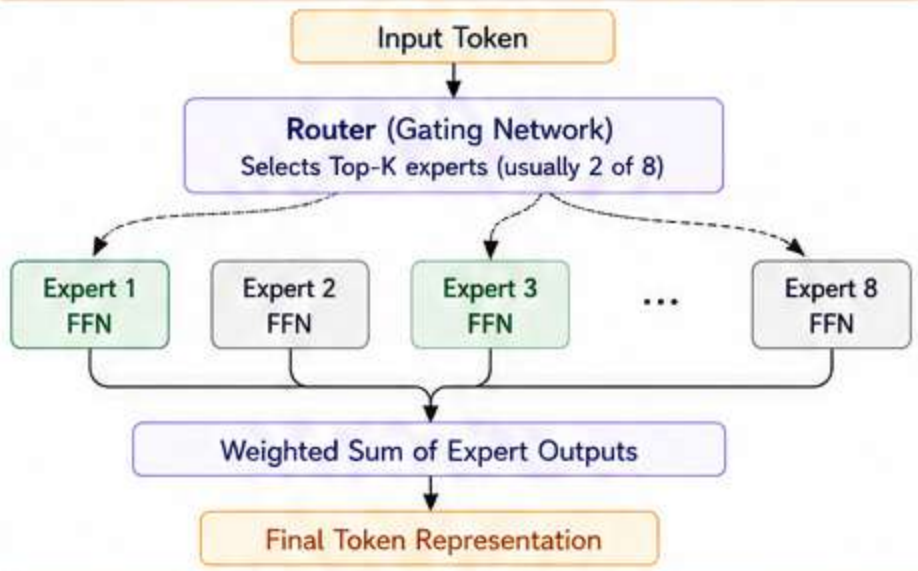
3 ENCODER-DECODER MODELS

- Encoder reads the input, decoder generates the output
- Ideal for tasks where output differs from input
- Text-to-text framework (T5 popularized this)

Model	Params	Best At
T5	60M–11B	Text-to-text; summarization, QA, translation
BART	140M–400M	Summarization, denoising, generation
mT5	300M–13B	Multilingual text-to-text
NLLB	54M–54B	200-language translation (Meta)
Flan-T5	60M–11B	Instruction-tuned T5

4 MIXTURE OF EXPERTS (MoE)

Instead of every token passing through all FFN weights, MoE routes each token to a small subset of "expert" FFN layers.



Model	Total Params	Active Params	Experts
Mixtral 8×7B	47B	12.9B	8 (top-2)
DeepSeek V3	671B	37B	256 (top-8)
Grok-1	314B	~86B	8 (top-2)

Key benefit: Inference cost of a 12B model with the capacity of a 47B model.

QUICK COMPARISON SUMMARY

Aspect	Encoder-Only	Decoder-Only	Encoder-Decoder
Attention	Bidirectional (no mask)	Causal (look-left only)	Encoder: bidirectional Decoder: causal + cross-attention
Primary Use	Understanding	Generation	Transform input → output
Examples	BERT, RoBERTa, ALBERT, DeBERTa, ModernBERT	GPT, LLaMA, Mistral, Mixtral, Qwen, DeepSeek	T5, BART, mT5, NLLB, Flan-T5
Strength	Best for classification, NER, retrieval, embeddings	Best for chat, coding, reasoning, creative generation	Best for translation, summarization, structured output
Limitation	Not for generation	Limited understanding compared to encoder models	More complex, higher compute

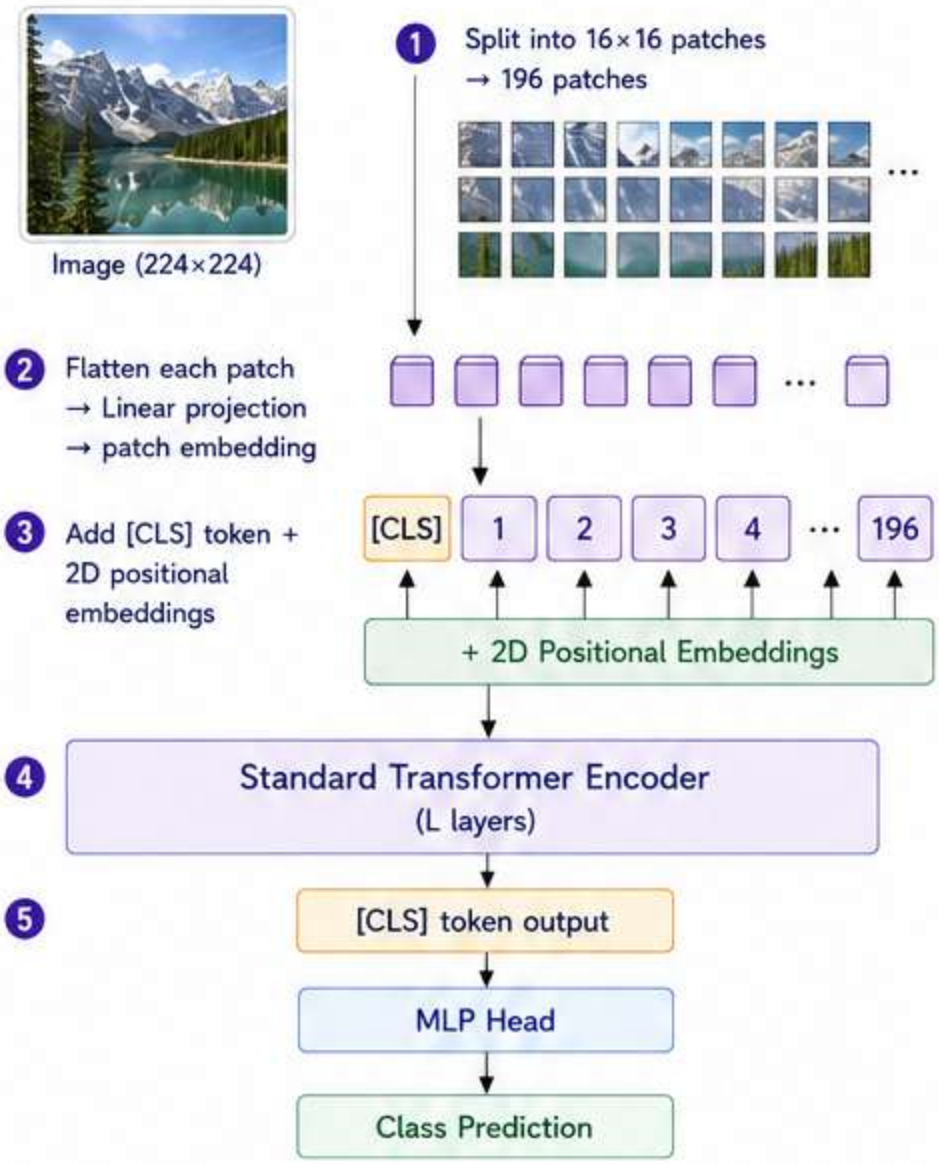
KEY TAKEAWAYS

- Different architectures excel at different tasks.
- MoE models deliver more capacity without proportional compute.
- Long context is an active area of research and engineering.
- The best models combine better objectives, data, scaling, and systems innovations.

VISION TRANSFORMERS & MULTIMODAL

1 VISION TRANSFORMER (VIT)

ViT applies the same Transformer architecture to images by treating image patches as tokens.

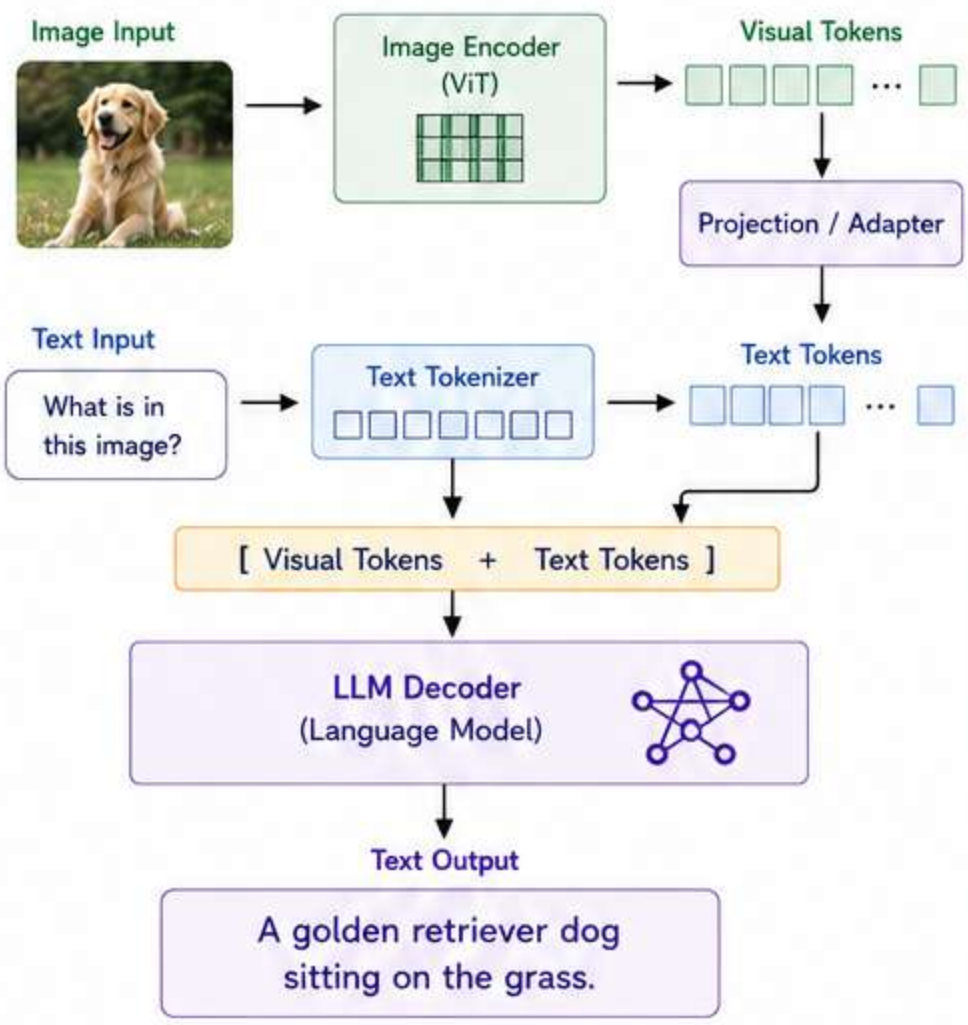


2 VISION TRANSFORMER MODELS

Model	Params	Patch Size	Innovation
ViT-B/16	86M	16 × 16	Original image Transformer
ViT-L/16	307M	16 × 16	Larger version
DeiT	86M	16 × 16	No extra data needed, distillation
DINO / DINOv2	86M–307M	14 × 14	Self-supervised, excellent features
SAM (Segment Anything)	632M	16 × 16	ViT encoder for segmentation
SigLIP	400M	varies	Better CLIP alternative from Google

ViT proved that Transformers are not just for language — they are a general-purpose architecture.

3 MULTIMODAL TRANSFORMERS



Model	Vision Encoder	Language Model	Capability
LLaVA	CLIP ViT-L	LLaMA / Mistral	Open-source VLM
Flamingo	NFNet + Perceiver	Chinchilla	Few-shot vision-language
GPT-4V	Unknown	GPT-4	State-of-the-art VLM
Gemini	Custom	Gemini	Native multimodal from day 1
PaliGemma	SigLIP ViT	Gemma	Google open VLM

Multimodal Transformers unify vision, language, and beyond.

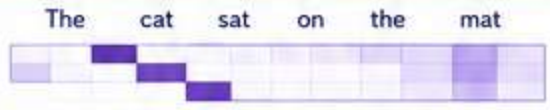
The future is multi-input, multi-output.



✦ ATTENTION HEAD ANALYSIS & INTERPRETABILITY ✦

1 WHAT DO ATTENTION HEADS LEARN?

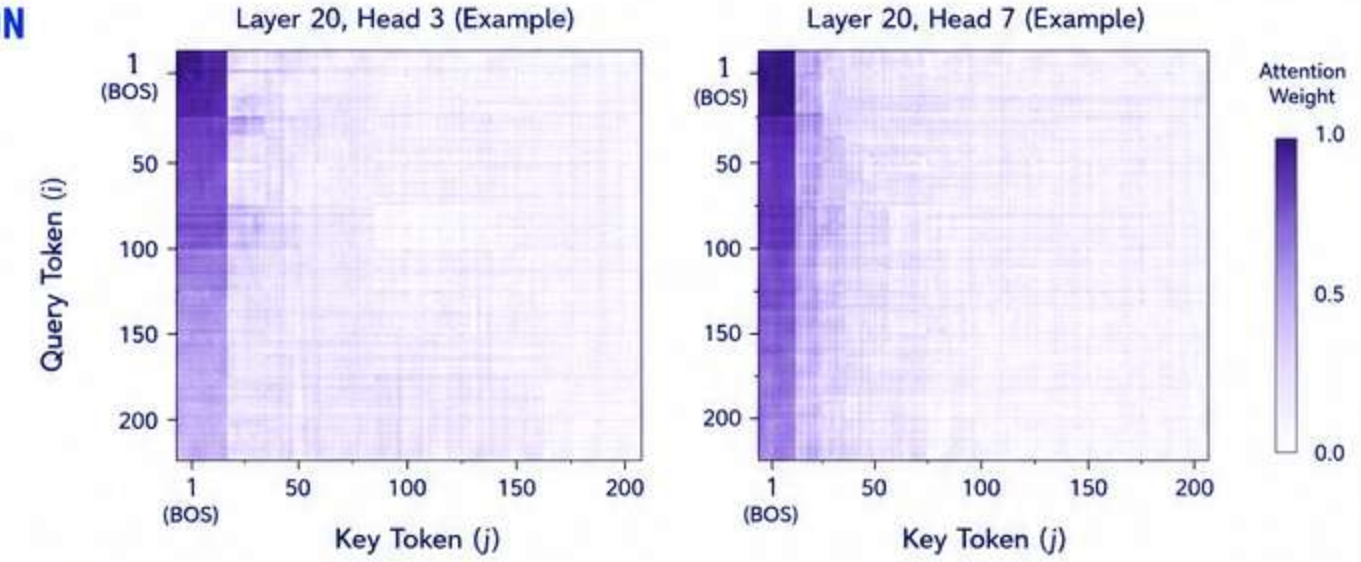
Research into what individual attention heads specialize in:

Head Type	What it Attends To	Example	Visualization (Attention Pattern)
Positional heads	Previous or next token (local syntax)	The [cat] sat on the mat	
Syntactic heads	Subject-verb, noun-adjective dependencies	The [cat] [chased] the [mouse]	
Coreference heads	Pronouns → referred noun entities	[Alice] said [she] would win	
Rare word heads	Attend to unusual tokens	The quizzical [zebroid] perplexed everyone	
BOS heads	Attend heavily to [BOS] token (no-op attention)	[BOS] The cat sat on the mat	
Semantic heads	Thematically related words across distance	The [doctor] treated the patient in the [hospital]	
Copy heads	Repeat/copy content from earlier in context	Answer: [42]. The answer is [42].	

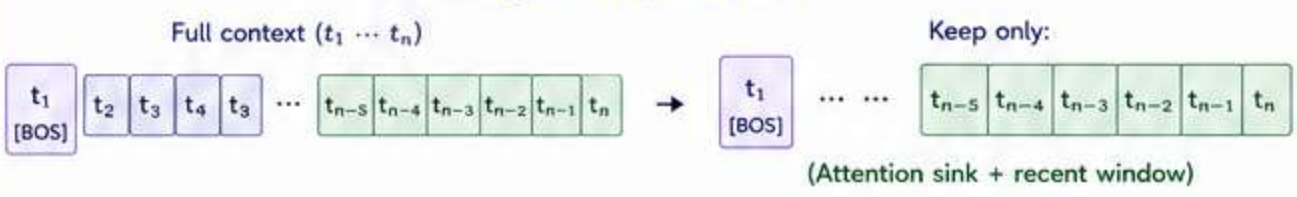
Low attention — High attention Darker color = higher attention weight

2 ATTENTION SINK PHENOMENON

- In decoder models, a large fraction of attention mass concentrates on the first token (BOS) regardless of semantic relevance.
- First token acts as a "garbage collector" for attention that has no better target.
- StreamingLLM exploits this — keep only attention sinks + recent tokens → infinite context.
- Suggests attention is not purely semantic at all heads.



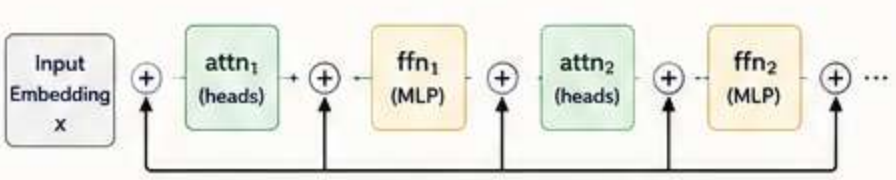
StreamingLLM (High-level idea)



3 TRANSFORMER CIRCUITS — MECHANISTIC INTERPRETABILITY

The circuits framework decomposes transformer behavior into interpretable components.

Residual Stream = $x + attn_1 + ffn_1 + attn_2 + ffn_2 + \cdots$



- Each attention head reads from and writes to the residual stream.
- Components compose via their effects on the stream.

Key findings from circuit analysis:

- Induction heads — 2-head circuit that enables in-context learning — A B A B ? !
- Copy suppression heads — prevent verbatim repetition — The cat sat. The [cat] sat.
- Name mover heads — move entity names to prediction position — John went to Paris. [John] [MASK]
- FFN layers store factual associations like key-value memories — Paris → is capital of → France

Mechanistic interpretability turns black-box models into understandable systems — revealing how and why transformers work.

MODERN ARCHITECTURAL IMPROVEMENTS

1 MODERN LLM ARCHITECTURE CHEAT SHEET

Every modern LLM (LLaMA 3, Mistral, Gemma, Qwen) has evolved from the original transformer with these improvements:

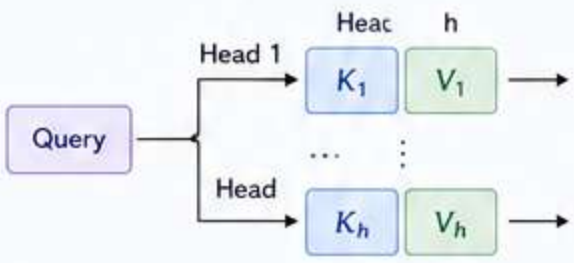
Component	Original (2017)	Modern Standard (2024)
Positional Encoding	Sinusoidal (absolute)	RoPE (relative, rotary)
Normalization	Post-Norm LayerNorm	Pre-Norm RMSNorm
Activation	ReLU	SwiGLU
Attention	MHA	GQA (Grouped Query Attention)
FFN ratio	$4 \times d_{\text{model}}$	$\sim 2.67 \times d_{\text{model}}$ (SwiGLU has 3 matrices)
Bias terms	Yes	No bias in linear layers
Vocab size	30K	128K+ (tiktoken, SentencePiece)
Context	512 tokens	128K – 1M tokens
Precision	FP32	BF16 + Flash Attention

2 MULTI-LATENT ATTENTION (MLA) — DEEPSEEK INNOVATION

DeepSeek V2/V3 introduced MLA to reduce KV cache memory:

Standard MHA

Cache K and V for all heads separately → large KV cache

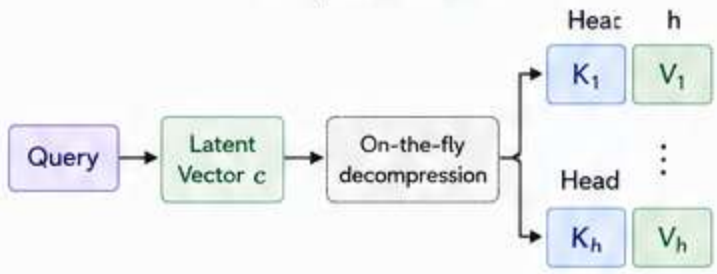


Cache size: $O(h \cdot n \cdot d_{\text{head}})$

MLA (DeepSeek)

Compress K and V into a low-rank latent vector c

At inference time: decompress $c \rightarrow K, V$ as needed



Cache size: $O(r \cdot n \cdot d_{\text{model}}),\ r \ll h \cdot d_{\text{head}}$

Why it works

- Most of the information in K, V lives in a low-dimensional subspace.
- MLA stores only a compact latent vector c.
- Reconstruct K, V from c only when needed.
- Massive memory savings, minimal quality loss.

Result: 93% KV cache reduction vs MHA
Better quality than MQA/GQA

3 STATE SPACE MODELS — TRANSFORMER ALTERNATIVE

A new class of architectures challenging transformers for long sequences:

Model	Type	Key Property	Notes / Strengths
Mamba	SSM	Linear complexity, strong on long sequences	Selective state updates, hardware efficient
Mamba-2	SSM + Attention	Structured State Space Duality	Better expressive power, maintains linear scaling
RWKV	RNN + Attention	Linear time, parallelizable like transformer	No recurrence at inference, efficient training
RetNet	Retention mechanism	$O(1)$ inference, $O(n)$ training	Retention (instead of attention) for long-context
Jamba	Mamba + Transformer	Hybrid: best of both worlds	Strong performance across many tasks

Why SSMs matter

- Linear or sub-quadratic complexity → scales to millions of tokens
- Constant memory at inference (e.g., RetNet)
- Competitive quality on long-context benchmarks

Transformers still lead on most benchmarks, but SSMs are competitive on very long sequences.
The future may be hybrid: Attention + SSM.

TOKENIZATION

1 HOW TEXT BECOMES TOKENS

Raw text: `"Transformers are amazing!"`

↓

Tokenized: `"Transform"` | `"ers"` | `" are"` | `" amaz"` | `"ing"` | `"!"`

↓

Token IDs: `8291` | `364` | `389` | `6381` | `278` | `0`

2 TOKENIZATION ALGORITHMS

Algorithm	How it Works	Used In	Notes
BPE (Byte Pair Encoding)	Merge most frequent byte pairs iteratively	GPT-2, GPT-3, LLaMA	Simple, effective, widely used
WordPiece	Like BPE but uses likelihood not frequency	BERT, DistilBERT	Better for understanding tasks (MLM)
SentencePiece	Language-agnostic BPE/Unigram on raw text	LLaMA, T5, Gemma	No pre-tokenization (works on raw Unicode text)
Tiktoken	Fast BPE with byte-level fallback	GPT-4, GPT-4o, Claude	Highly optimized, production-grade
Unigram LM	Keep tokens that minimize vocab perplexity	Some multilingual models	Probabilistic, prunes unnecessary tokens

3 TOKENIZATION EDGE CASES

Numbers:	`"1234567890" → ["123", "456", "789", "0"]` (bad for math)
Spaces:	`" hello" ≠ "hello"` (leading space is part of the token)
Case:	`"cat" ≠ "Cat" ≠ "CAT"` (may be different tokens)
Code:	Indentation encoded as spaces (token expensive)
Unicode:	Rare chars fall back to byte-level tokens (longer sequences)
Languages:	English is most token-efficient; Chinese/Hindi use more tokens per word

4 VOCABULARY SIZE TRADE-OFFS

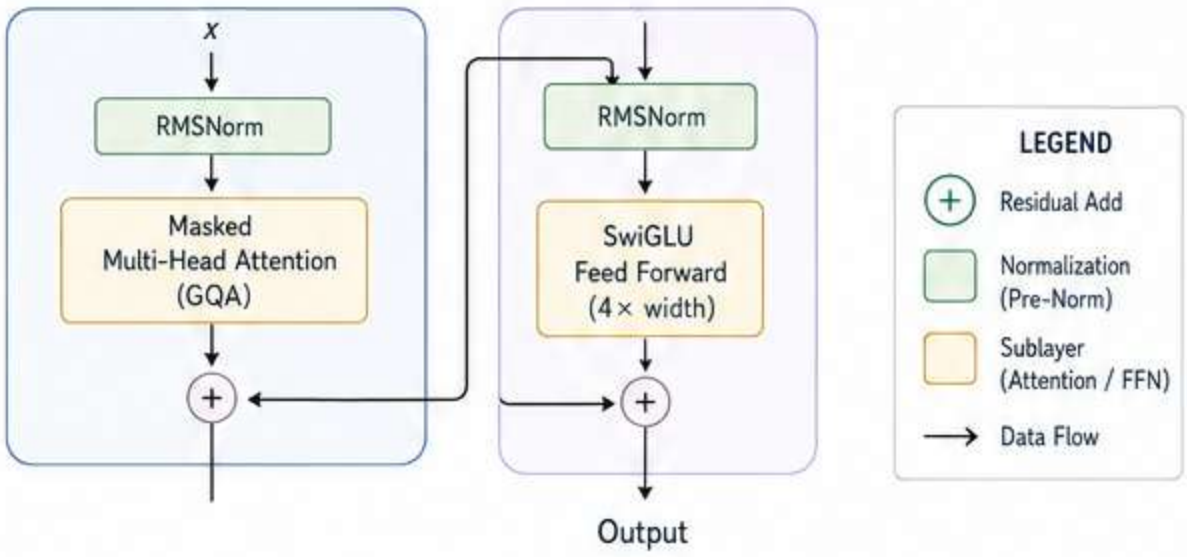
Vocab Size	Pros	Cons	Best For
Small (8K–16K)	• Small embedding table • Fast, memory efficient	• Longer sequences • More tokens per word	Resource-constrained models, edge devices
Medium (32K–64K)	• Good balance • Efficient for most languages	• Some rare words split	Most modern LLMs (Standard choice)
Large (128K–256K)	• Shorter sequences • Better for multilingual	• Large embedding matrix • More parameters & memory	Multilingual, code, long-context specialized models

Tokenization is the foundation. Better tokenization → better efficiency, lower cost, better performance.

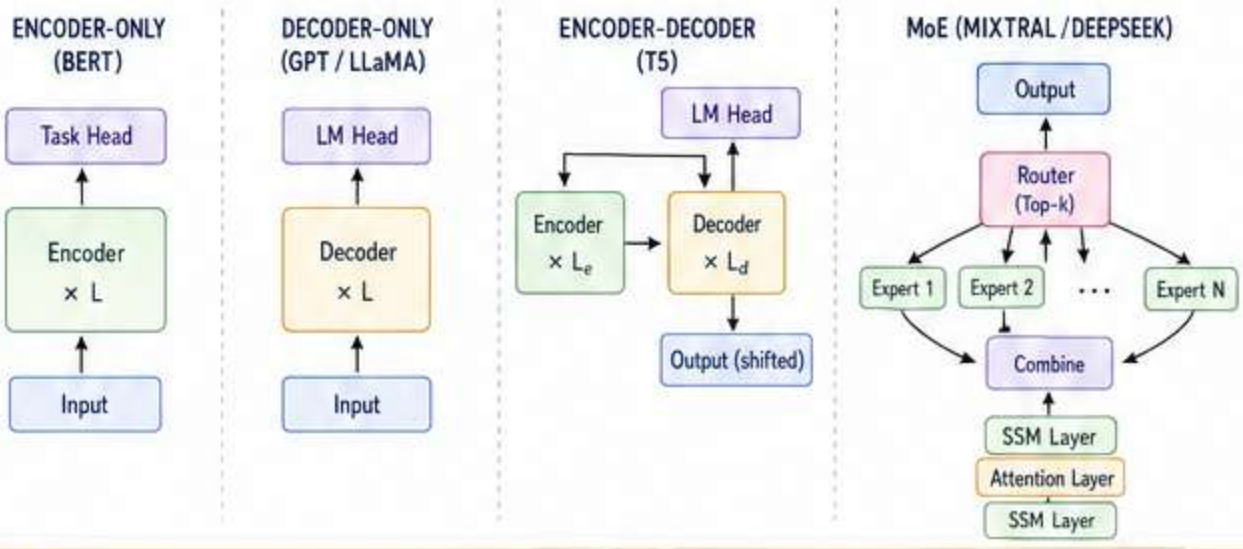
TRANSFORMER COMPLETE REFERENCE — ONE PAGE

CORE FORMULA	$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) \cdot V$
MULTI-HEAD	Run h attention heads → Concat → Project
MASKING	Causal mask: $-\infty$ for future positions in decoder
RESIDUAL	$\text{output} = \text{LayerNorm}(x + \text{Sublayer}(x))$
NORM	RMSNorm (modern) / LayerNorm (original) — Pre-Norm preferred
FFN	$\text{SwiGLU}(xW_1) \cdot (xW_2) \cdot W_3$ — $4\times$ width, per-token
POSITION	RoPE (modern) / Sinusoidal (original) / ALiBi
ATTENTION TYPES	Self · Masked-Self · Cross · GQA · MQA · MLA
CONTEXT	Flash Attention · Sliding Window · YaRN · Ring Attention
TRAINING OBJ	CLM (decoder) / MLM (encoder) / Span Corruption (enc-dec)
OPTIMIZER	AdamW $\beta_1=0.9$ $\beta_2=0.95$ + cosine LR decay + warmup
PRECISION	BF16 forward + FP32 master weights
SCALING	Chinchilla: 20 tokens per parameter optimal
VARIANTS	Encoder-Only / Decoder-Only / Enc-Dec / MoE / SSM Hybrid
VISION	ViT: image → patches → tokens → encoder → CLS → class
MODERN STACK	RoPE + RMSNorm + SwiGLU + GQA + Pre-Norm + No Bias

STANDARD TRANSFORMER BLOCK (DECODER)



COMMON VARIANTS AT A GLANCE



KEY PAPERS TO READ

- **2017** — **Attention Is All You Need** (Vaswani et al.) — The original
- **2018** — **BERT** (Devlin et al.) — Encoder pre-training
- **2019** — **GPT-2** (Radford et al.) — Language models as zero-shot learners
- **2020** — **Scaling Laws** (Kaplan et al.) — Power laws for neural LMs
- **2020** — **GPT-3** (Brown et al.) — Few-shot learning emergent abilities
- **2021** — **ViT** (Dosovitskiy et al.) — Transformers for images
- **2022** — **Chinchilla** (Hoffmann et al.) — Compute-optimal training
- **2022** — **Flash Attention** (Dao et al.) — IO-aware exact attention
- **2023** — **LLaMA** (Touvron et al.) — Open foundation models
- **2023** — **Mistral** (Jiang et al.) — GQA + Sliding Window
- **2023** — **Flash Attention 2** (Dao) — 2× faster, better parallelism
- **2024** — **GQA** (Ainslie et al.) — Grouped query for long context
- **2024** — **DeepSeek V2** (DeepSeek) — MLA + MoE at scale
- **2024** — **Mamba** (Gu & Dao) — Linear SSM challenging transformers

Stay curious. Read papers. Build. Ship. Iterate.

TRANSFORMERS ARE THE PLATFORM. INNOVATION IS IN THE DETAILS.

Better architectures + more data + more compute = better models.